

Programming 1 — Formative Project (Week 7)

Weighting: 50 % of module grade (formative)

Mode: Individual Submission

Language: Python 3

Project Overview

In this project, you'll build a Student Grade/Assignment Tracker (runs on the command line) that allows users to record homework and exam results, view and filter assignments, and display grade summaries within a single terminal session. This project specification will be available in the repository (<https://github.com/navin-alche/ProgrammingOneFormativeAssignment.git>). Students will be required to clone this repository on their local machine.

Project Requirements

Your program must:

1. Add Assignments
 - Accept subject, title, score, max score, due date, and assignment type (homework or exam).
 - Store the assignments in a list or dictionary.
2. List Assignments
 - Display all assignments in a clean, readable format.
3. Filter Assignments
 - Filter by type (homework/exam), subject, or month (e.g., 2025-10).
4. Summarize Grades
 - Display overall average, per-subject averages, and highest/lowest scoring assignment.
5. Validate Input
 - Handle invalid menu choices and scores gracefully.
6. Session Only
 - All data remains in memory during execution. No saving or loading to files.

Technical Requirements

- Strings & Conditionals: For menu control, validation, and flow.
- Loops: Main program loop and assignment iteration.
- Functions: Break down features logically (e.g., `add_homework()`, `filter_assignments()`, `show_summary()`).
- Collections: Use lists or dictionaries for assignments and summaries.
- OOP:
 - Assignment class (attributes: `subject`, `title`, `score`, `max_score`, `due_date`, `type`).
 - GradeTracker class (methods for `add/list/filter/summary`).
- Inheritance: Implement Homework and Exam as subclasses of Assignment.
- Robustness: Input validation (e.g., numeric score, menu selection), graceful handling of empty datasets.

Sample Menu

- 1) Add homework
- 2) Add exam
- 3) List assignments
- 4) Filter (by subject / type / month)
- 5) Show summary
- 0) Exit

Deliverables

- GitHub repository with regular commits
- Python source code (.py files)
- README.md (max 2 pages) with:
 - Project overview & features
 - Instructions to run the program
 - Menu structure
 - Sample interactions
- Screenshots showing add, list, filter, and summary
- Short reflection (max 1 page - pdf):
 - What did you learn?
 - Challenges you faced?
 - How do you intend to improve it, given more time?

Suggested Class Skeleton

e.g class Assignment

```
class Assignment:
    def __init__(self, subject, title, score, max_score, due_date, atype):
        self.subject = subject.lower().strip()
        self.title = title
        self.score = float(score)
        self.max_score = float(max_score)
        self.due_date = due_date
        self.type = atype # 'homework' or 'exam'
```

Expectations

- Clean, user-friendly menu
- Program runs without errors or crashes
- Logical structure with functions and OOP
- Inheritance applied purposefully
- Proper Git history — no single final commit dump

Optional Features

- Grade threshold / low-score warnings
- Top performing subjects
- Undo last entry
- Basic test assertions

Assessment Rubric (50 %)

Section	Detailed Criteria	Weight
1. Environment & Setup	• Git environment correctly configured and repository cloned from GitHub. • Initial setup tested with a working print statement. • At least 3+ incremental commits showing progressive development (not a single final push). • Clear folder structure (main.py, supporting files).	8%
2. Strings & Conditionals	• Menu system implemented with clear, readable text and consistent structure. • User inputs handled using input(); appropriate use of string formatting for output messages. • Conditional statements (if/elif/else) used to navigate between menu options correctly. • Invalid inputs handled gracefully with appropriate feedback messages.	8%
3. Functions & Modularity	• Code divided logically into functions that each perform a single task (e.g., add_homework(), add_exam(), show_summary()). • Functions use parameters and return values effectively instead of relying solely on global variables. • Demonstrates understanding of code reuse and avoids repetition. • Each function is well-commented and contributes to an overall modular program flow.	8%
4. Loops & Collections	• Main program loop correctly manages user interactions until exit option is chosen. • Proper use of for or while loops for iterating through data collections. • Assignments stored in appropriate data structures (e.g., list of dictionaries or list of class objects). • Operations such as filtering or calculating averages are implemented using loops efficiently and without logic errors.	8%
5. Classes (OOP)	• Assignment class correctly defined with attributes (subject, title, score, max_score, due_date, type). • GradeTracker class implemented with methods to add, list, filter, and summarize assignments. • Evidence of understanding encapsulation—attributes and methods logically grouped. • Demonstrates correct object creation and method calls. • Code is readable and aligns with Python naming conventions.	9%
6. Inheritance & Correctness	• Homework and Exam subclasses extend Assignment and correctly inherit properties. • Appropriate use of super() constructor to initialize inherited attributes. • Program runs reliably without crashes during all menu operations. • Input validation ensures that only valid assignments are stored (e.g., score not exceeding max_score). • Output values and summaries are logically correct (e.g., average = total score / total max score).	9%

TOTAL: 50%